DATA STRUCTURES
Viva / Oral Question Bank

Short answers, code routines and comparison tables for exam and lab viva preparation

Unit	Topic	Questions
1	Introduction and Lists	31
2	Stacks and Queues	31
3	Trees	28
4	Graphs	27
5	Sorting, Searching and Hashing	24
	Total	**141**

All code is written in C. Complexities are given for the usual textbook implementations.

PDF BY VL







✨ WHY JOIN ?

🏆 Trusted by 80,000+ Engineering Students
🔔 Stay Updated with latest university news
🚀 Make your engineering journey easy with us

Best Regards, Vishwajeet Londhe

© 2026 Vishwajeet Londhe. All Rights Reserved.

UNIT 1 - Introduction and Lists

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>What is a data structure?</td><td>A data structure is a particular way of organising and storing data in memory so that it can be accessed and modified efficiently.
It captures two things: the data items themselves and the relationships between them (for example, 'next item', 'parent of', 'connected to').</td></tr>
<tr><td>2</td><td>Why do we need data structures?</td><td>• To handle large amounts of data efficiently (fast search, insert and delete).
• To write reusable code: a well-built structure such as a stack can be used in many programs.
• To model real-world relationships, such as a road map (graph) or a file system (tree).
• To choose the right trade-off between time and memory for a given problem.</td></tr>
<tr><td>3</td><td>How are data structures classified?</td><td><table><tr><th>Category</th><th>Meaning</th><th>Examples</th></tr><tr><td>Primitive</td><td>Basic types directly supported by the machine / language</td><td>int, char, float, double, pointer</td></tr><tr><td>Non-primitive, linear</td><td>Elements form a sequence; each has one predecessor and one successor</td><td>Array, linked list, stack, queue</td></tr><tr><td>Non-primitive, non-linear</td><td>Elements form a hierarchy or network</td><td>Tree, graph</td></tr></table></td></tr>
<tr><td>4</td><td>Differentiate between linear and non-linear data structures.</td><td><table><tr><th>Linear</th><th>Non-linear</th></tr><tr><td>Elements are stored in a sequence</td><td>Elements are stored hierarchically or as a network</td></tr><tr><td>Each element has at most one predecessor and one successor</td><td>An element can be linked to many others</td></tr><tr><td>Can be traversed in a single pass</td><td>Usually needs a traversal strategy (DFS, BFS, inorder, ...)</td></tr><tr><td>Simpler to implement</td><td>More complex to implement</td></tr><tr><td>Array, linked list, stack, queue</td><td>Tree, graph</td></tr></table></td></tr>
<tr><td>5</td><td>What is an algorithm? List its main characteristics.</td><td>An algorithm is a finite, step-by-step procedure for solving a problem.
• Input: zero or more well-defined inputs.
• Output: at least one result.
• Definiteness: every step is clear and unambiguous.
• Finiteness: it terminates after a finite number of steps.
• Effectiveness: each step is simple enough to be carried out.</td></tr>
<tr><td>6</td><td>What is time complexity and space complexity? What is Big-O notation?</td><td>Time complexity describes how the running time grows with input size n. Space complexity describes how the extra memory needed grows with n.
Big-O gives an upper bound on the growth rate, ignoring constants and lower-order terms. Common orders, from fastest to slowest: O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n).</td></tr>
<tr><td>7</td><td>Define an Abstract Data Type (ADT).</td><td>An ADT is a mathematical model of a data type that is defined by what operations it supports and what they mean, not by how they are coded. The user sees the interface only; the implementation is hidden.
Example: the Stack ADT is defined by push, pop, peek and isEmpty. It may be implemented with an array or with a linked list, and the user's program does not change.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>8</td><td>What is the difference between a data structure and an ADT?</td><td>
<table>
<tr><th>ADT</th><th>Data structure</th></tr>
<tr><td>Logical view: what operations are available</td><td>Physical view: how the data is laid out in memory</td></tr>
<tr><td>Language independent</td><td>Implemented in a specific language</td></tr>
<tr><td>Example: List, Stack, Queue (as interfaces)</td><td>Example: array-based list, linked list</td></tr>
</table>
</td></tr>
<tr><td>9</td><td>List the benefits of using ADTs.</td><td>

Modularity: the program is divided into small, independent parts that are easy to debug and maintain.
Reusability: operations are written once and used in many places.
Information hiding: the implementation can be changed (say, array to linked list) without touching the code that uses the ADT.
Code is easier to read and reason about.

</td></tr>
<tr><td>10</td><td>Define the List ADT and list its common operations.</td><td>A list is a finite sequence of elements A_1, A_2, ..., A_N of the same type, where order matters. A list with N = 0 is the empty list.

Insert an element (at beginning, at end, or at a given position)
Delete an element (by position or by value)
Search / Find an element
Traverse / Print the list
FindKth, Next, Previous, Length, Merge

</td></tr>
<tr><td>11</td><td>In how many ways can a list be implemented?</td><td>

Array implementation - elements stored in contiguous memory.
Linked-list implementation - elements stored in nodes joined by pointers (singly, doubly, circular).
Cursor implementation - linked list simulated with an array of structures and integer indices, for languages without pointers.

</td></tr>
<tr><td>12</td><td>State the merits and demerits of the array implementation of a list.</td><td>
<table>
<tr><th>Merits</th><th>Demerits</th></tr>
<tr><td>Direct (random) access to the k-th element in O(1)</td><td>Size must be fixed in advance; may waste memory or overflow</td></tr>
<tr><td>No extra memory for pointers</td><td>Insertion and deletion in the middle need shifting of elements: O(n)</td></tr>
<tr><td>Traversal is cache friendly and fast</td><td>Resizing needs allocating a new array and copying</td></tr>
<tr><td>Binary search is possible on a sorted array</td><td></td></tr>
</table>
</td></tr>
<tr><td>13</td><td>What is a linked list? Give the node structure.</td><td>A linked list is a collection of nodes; each node holds a data value and a pointer to the next node. Nodes can be anywhere in memory, so the list grows and shrinks at run time. The first node is reached through a pointer called head; the last node's pointer is NULL.
<pre><code>struct Node {
    int data;
    struct Node *next;
};</code></pre>
</td></tr>
<tr><td>14</td><td>What are the different types of linked lists?</td><td>

Singly linked list - each node has one pointer (next).
Doubly linked list - each node has two pointers (prev and next).
Circular linked list - the last node points back to the first node (singly or doubly).

</td></tr>
<tr><td>15</td><td>Describe a singly linked list.</td><td>Each node has two fields: data and next. Traversal is possible in one direction only, from head to NULL.
<pre><code>head -> [10|*] -> [20|*] -> [30|*] -> NULL</code></pre>
</td></tr>
</table>

S.No	Question	Answer

16	**Describe a doubly linked list and its node structure.**	Each node has three fields: a pointer to the previous node, the data, and a pointer to the next node. It can be traversed both forwards and backwards, and a node can be deleted when only a pointer to that node is known.
		`struct DNode { struct DNode *prev; int data; struct DNode *next; };`
		`head -> [prev|10|next] <-> [prev|20|next] <-> [prev|30|next] (first prev = NULL, last next = NULL)`
		Cost: one extra pointer per node, and insertion/deletion must update more links.

17	**What is a circular linked list? When does a singly or doubly linked list become circular?**	In a circular linked list no node has a NULL pointer. In a **singly** list it becomes circular when the **last node's next** points to the first node. In a **doubly** list it becomes circular when the last node's next points to the first node **and** the first node's prev points to the last node. `head -> [3|*] -> [5|*] -> [9|*] ^_____|` Use: round-robin CPU scheduling, music playlists on repeat, buffers. Traversal must stop when we return to the starting node, otherwise it loops forever.
18	**What is a header (dummy) node? Why is it used?**	A header node is a special node placed before the first real data node. It does not hold user data (or may hold bookkeeping such as the count). Because the list always has this node, **insertion and deletion at the front need no special case**, and the code for an empty list is the same as for a non-empty list.
19	**What is the cursor implementation of a linked list? Where is it used?**	The list is simulated with a **global array of structures**. Each structure holds a data field and an integer **index** (the cursor) of the next structure instead of a pointer. A free list of unused slots is maintained, and a function like `CursorAlloc` takes a slot from it. It is used in languages that do not support pointers (early BASIC, FORTRAN) and when memory must be managed in a fixed block.
20	**Write a function to insert a node at the beginning of a singly linked list.**	`struct Node *insertFront(struct Node *head, int x) { struct Node *n = malloc(sizeof(struct Node)); if (n == NULL) { printf("Out of space\n"); return head; } n->data = x; n->next = head; /* new node points to old first node */ return n; /* new node becomes head */ }` Time: O(1).
21	**Write a function to insert a node after a given node P.**	`void insertAfter(struct Node *p, int x) { if (p == NULL) return; struct Node *n = malloc(sizeof(struct Node)); if (n == NULL) { printf("Out of space\n"); return; } n->data = x; n->next = p->next; /* step 1: link new node forward */ p->next = n; /* step 2: link previous to new */ }` The order of the two pointer assignments matters; reversing them loses the rest of the list. Time: O(1).

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>22</td><td>Write a function to delete the first node containing a given value.</td><td><pre>
struct Node *deleteValue(struct Node *head, int x) {
    struct Node *cur = head, *prev = NULL;
    while (cur != NULL && cur->data != x) {
        prev = cur;
        cur = cur->next;
    }
    if (cur == NULL) return head;   /* not found */
    if (prev == NULL) head = cur->next; /* first node */
    else prev->next = cur->next;
    free(cur);
    return head;
}
</pre>Time: O(n) to search, O(1) to unlink.</td></tr>
<tr><td>23</td><td>Write a function to reverse a singly linked list.</td><td><pre>
struct Node *reverse(struct Node *head) {
    struct Node *prev = NULL, *cur = head, *nxt;
    while (cur != NULL) {
        nxt = cur->next;   /* remember the rest   */
        cur->next = prev; /* flip the link   */
        prev = cur;
        cur = nxt;
    }
    return prev;   /* new head */
}
</pre>Time O(n), extra space O(1).</td></tr>
<tr><td>24</td><td>How can you find the middle node of a linked list in one pass?</td><td>Use two pointers, slow and fast. Move slow by one node and fast by two nodes in each step. When fast reaches the end, slow is at the middle.</td></tr>
<tr><td>25</td><td>How can you detect a loop (cycle) in a linked list?</td><td>Floyd's tortoise-and-hare method: move one pointer by one step and another by two steps. If the list has a cycle the two pointers will eventually meet; if the fast pointer reaches NULL there is no cycle. Time O(n), space O(1).</td></tr>
<tr><td>26</td><td>How are polynomials represented using a linked list?</td><td>Each term is a node holding the coefficient, the exponent and a link to the next term. Terms are kept in decreasing order of exponent, which makes addition of two polynomials a simple merge.<pre>
struct Term {
    int coeff;
    int exp;
    struct Term *next;
};

5x^3 + 2x + 7 => [5|3] -> [2|1] -> [7|0] -> NULL
</pre></td></tr>
<tr><td>27</td><td>Compare the running times of common operations on an array and a linked list.</td><td><table>
<tr><th>Operation</th><th>Array</th><th>Singly linked list</th></tr>
<tr><td>Access k-th element</td><td>O(1)</td><td>O(n)</td></tr>
<tr><td>Insert at beginning</td><td>O(n)</td><td>O(1)</td></tr>
<tr><td>Insert at end</td><td>O(1) if space is free</td><td>O(n), or O(1) with a tail pointer</td></tr>
<tr><td>Insert / delete after a known position</td><td>O(n)</td><td>O(1)</td></tr>
<tr><td>Search (unsorted)</td><td>O(n)</td><td>O(n)</td></tr>
<tr><td>Search (sorted)</td><td>O(log n) by binary search</td><td>O(n)</td></tr>
</table></td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>28</td><td>Give the advantages and disadvantages of a linked list.</td><td>
<table>
<tr><th>Advantages</th><th>Disadvantages</th></tr>
<tr><td>Dynamic size; grows and shrinks at run time</td><td>No random access; reaching the k-th node takes O(n)</td></tr>
<tr><td>Insertion and deletion need no shifting of elements</td><td>Extra memory for the pointer in every node</td></tr>
<tr><td>No memory is reserved in advance, so little is wasted</td><td>Poor cache locality; nodes are scattered in memory</td></tr>
<tr><td>Easy to build stacks, queues, graphs on top of it</td><td>A singly linked list cannot be traversed backwards</td></tr>
</table>
</td></tr>
<tr><td>29</td><td>Differentiate between an array and a linked list.</td><td>
<table>
<tr><th>Array</th><th>Linked list</th></tr>
<tr><td>Contiguous memory</td><td>Non-contiguous memory</td></tr>
<tr><td>Fixed size (static) in most languages</td><td>Size changes at run time</td></tr>
<tr><td>Index-based random access</td><td>Sequential access only</td></tr>
<tr><td>Insert/delete in the middle is costly</td><td>Insert/delete is cheap once position is known</td></tr>
<tr><td>No pointer overhead</td><td>Pointer overhead per node</td></tr>
<tr><td>Binary search is possible</td><td>Binary search is not practical</td></tr>
</table>
</td></tr>
<tr><td>30</td><td>Why is binary search not suitable for a linked list?</td><td>Binary search needs to jump to the middle element in O(1). In a linked list the middle can only be reached by walking from the head, which costs O(n) for every step, so the total cost is no better than linear search.</td></tr>
<tr><td>31</td><td>List some applications of lists.</td><td>

Polynomial arithmetic and sparse matrices.
Implementation of stacks, queues, hash-table chaining and graph adjacency lists.
Memory management: the operating system keeps a free list of unused blocks.
File systems such as FAT keep the blocks of a file as a chain of links.
Undo/redo and browser history (doubly linked list); round-robin scheduling (circular list).

</td></tr>
</table>

UNIT 2 - Stacks and Queues

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>Define a stack.</td><td>A stack is a linear data structure in which insertion and deletion are done only at one end, called the top. The element added last is removed first, so it follows the LIFO (Last In, First Out) principle. Think of a pile of plates.</td></tr>
<tr><td>2</td><td>List the basic operations of a stack.</td><td>• push(x) - place x on the top.
• pop() - remove and return the top element.
• peek() / top() - return the top element without removing it.
• isEmpty() - true if the stack has no elements.
• isFull() - true if an array-based stack has no free space.</td></tr>
<tr><td>3</td><td>What are stack overflow and stack underflow?</td><td>Overflow occurs when push is attempted on a full stack. Underflow occurs when pop (or peek) is attempted on an empty stack. Both must be checked before the operation.</td></tr>
<tr><td>4</td><td>Write the push and pop routines for an array-based stack.</td><td><pre>#define MAX 100
int stack[MAX];
int top = -1;   /* -1 means empty */

void push(int x) {
    if (top == MAX - 1) { printf("Stack overflow\n"); return; }
    stack[++top] = x;
}

int pop(void) {
    if (top == -1) { printf("Stack underflow\n"); return -1; }
    return stack[top--];
}</pre>Both operations are O(1).</td></tr>
<tr><td>5</td><td>Write the push and pop routines for a linked-list stack.</td><td><pre>struct Node { int data; struct Node *next; };
struct Node *top = NULL;

void push(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->next = top;   /* insert at the head */
    top = n;
}

int pop(void) {
    if (top == NULL) { printf("Stack underflow\n"); return -1; }
    struct Node *t = top;
    int x = t->data;
    top = top->next;   /* delete from the head */
    free(t);
    return x;
}</pre>Push and pop both work at the head of the list, so both are O(1). The stack never overflows until memory is exhausted.</td></tr>
<tr><td>6</td><td>Compare array and linked-list implementations of a stack.</td><td><table>
<tr><th>Array stack</th><th>Linked-list stack</th></tr>
<tr><td>Fixed capacity; overflow possible</td><td>Grows dynamically</td></tr>
<tr><td>No pointer overhead</td><td>One pointer per element</td></tr>
<tr><td>Better cache performance</td><td>Allocation cost for every push</td></tr>
<tr><td>Simple to code</td><td>Needs malloc/free handling</td></tr>
</table></td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>7</td><td>List the applications of a stack.</td><td>

Function-call management and recursion (call stack).
Conversion of infix expressions to postfix/prefix and evaluation of postfix expressions.
Checking balanced parentheses and matching tags.
Undo operations in editors and back button in browsers.
Depth-first search and backtracking (maze, N-Queens).
Reversing a string or a list.

</td></tr>
<tr><td>8</td><td>What are infix, prefix and postfix notations?</td><td>
<table>
<tr><th>Notation</th><th>Operator position</th><th>Example for A + B</th></tr>
<tr><td>Infix</td><td>Between the operands</td><td>A + B</td></tr>
<tr><td>Prefix (Polish)</td><td>Before the operands</td><td>+ A B</td></tr>
<tr><td>Postfix (Reverse Polish)</td><td>After the operands</td><td>A B +</td></tr>
</table>
</td></tr>
<tr><td>9</td><td>Why is postfix notation preferred by compilers?</td><td>Postfix needs no parentheses and no precedence rules; the order of the operators in the string is already the order of evaluation. It can be evaluated in a single left-to-right scan using one stack.</td></tr>
<tr><td>10</td><td>Write the algorithm to convert infix to postfix using a stack.</td><td>

Scan the infix expression from left to right.
If the symbol is an operand, append it to the output.
If it is (, push it on the stack.
If it is), pop operators to the output until (is found; discard the (.
If it is an operator, pop operators from the stack to the output while the top has higher precedence (or equal precedence and the operator is left-associative); then push the new operator.
At the end, pop all remaining operators to the output.

Precedence (high to low): ^ (right-associative), then * and /, then + and -.
</td></tr>
<tr><td>11</td><td>Convert (a+b)*(c+d)/f into postfix and prefix.</td><td>
<table>
<tr><th>Postfix</th><th>Prefix</th></tr>
<tr><td>a b + c d + * f /</td><td>/ * + a b + c d f</td></tr>
</table>
Because * and / have equal precedence and are left-associative, (a+b)*(c+d) is evaluated first and then divided by f.
</td></tr>
<tr><td>12</td><td>Convert A + B * (C - D) / (P - R) into postfix and prefix.</td><td>
<table>
<tr><th>Postfix</th><th>Prefix</th></tr>
<tr><td>A B C D - * P R - / +</td><td>+ A / * B - C D - P R</td></tr>
</table>
Steps: C-D and P-R are evaluated first (brackets), then B*(C-D), then the division, and finally the addition with A.
</td></tr>
<tr><td>13</td><td>Convert A - B + C - D into postfix.</td><td>The operators + and - have the same precedence and associate left to right, so the expression is ((A - B) + C) - D.
<pre>Postfix: A B - C + D -</pre>
</td></tr>
<tr><td>14</td><td>Evaluate the postfix expression 5 3 2 * + 4 -.</td><td>
<table>
<tr><th>Symbol</th><th>Action</th><th>Stack (bottom to top)</th></tr>
<tr><td>5</td><td>push</td><td>5</td></tr>
<tr><td>3</td><td>push</td><td>5, 3</td></tr>
<tr><td>2</td><td>push</td><td>5, 3, 2</td></tr>
<tr><td>*</td><td>pop 2 and 3, push 3*2</td><td>5, 6</td></tr>
<tr><td>+</td><td>pop 6 and 5, push 5+6</td><td>11</td></tr>
<tr><td>4</td><td>push</td><td>11, 4</td></tr>
<tr><td>-</td><td>pop 4 and 11, push 11-4</td><td>7</td></tr>
</table>
Result = 7. Note that for a non-commutative operator the second popped value is the left operand.
</td></tr>
</table>

S.No	Question	Answer
15	**How do you check whether the parentheses in an expression are balanced?**	1. Scan the expression from left to right. 2. On an opening bracket ([{, push it. 3. On a closing bracket, if the stack is empty or the top is not the matching opening bracket, the expression is **unbalanced**; otherwise pop. 4. At the end the expression is balanced only if the stack is empty.
16	**How is a stack used in recursion?**	Each function call creates an **activation record** (stack frame) containing the return address, parameters and local variables, and this frame is pushed on the call stack. When the function returns, its frame is popped and control goes back to the saved return address. Too-deep recursion causes stack overflow.
17	**What is the Tower of Hanoi problem? How many moves are needed?**	Move n discs from one peg to another using a third peg, moving one disc at a time and never placing a larger disc on a smaller one. The recursive solution moves n-1 discs to the helper peg, moves the largest disc, then moves n-1 discs onto it. Minimum number of moves = 2^n - 1 (for example, 3 discs need 7 moves).
18	**Define a queue.**	A queue is a linear data structure in which insertion takes place at one end, the **rear**, and deletion takes place at the other end, the **front**. The element inserted first is removed first, so it follows **FIFO** (First In, First Out), like people waiting in a line.
19	**List the operations of a queue.**	• **enqueue(x)** - add x at the rear. • **dequeue()** - remove and return the element at the front. • **peek() / front()** - read the front element without removing it. • **isEmpty()**, **isFull()** - status checks.

20	**Write the enqueue and dequeue routines for a simple array queue.**	`#define MAX 100`
		`int q[MAX];`
		`int front = 0, rear = -1;  /* empty when rear < front */`
		
		`void enqueue(int x) {`
		`  if (rear == MAX - 1) { printf("Queue overflow\n"); return; }`
		`  q[++rear] = x;`
		`}`
		
		`int dequeue(void) {`
		`  if (front > rear) { printf("Queue underflow\n"); return -1; }`
		`  return q[front++];`
		`}`
		Both are O(1).

21	**What is the drawback of a simple linear array queue?**	After several enqueue and dequeue operations, front moves forward and the vacated positions at the start of the array can never be reused, so the queue reports 'full' even though there is free space. This is called **false overflow**. A circular queue removes this problem.

22	**What is a circular queue? How are its full and empty conditions tested?**	A circular queue treats the array as a ring: after the last index, rear (or front) wraps around to index 0 using the modulus operator.
		`enqueue:  rear  = (rear  + 1) % MAX;`
		`dequeue:  front = (front + 1) % MAX;`
		A common convention keeps one slot unused: **empty** when front == rear, **full** when (rear + 1) % MAX == front. Alternatively, keep a separate count of elements.

S.No	Question	Answer

23	**Write the enqueue and dequeue routines for a circular queue.**	`int q[MAX], front = 0, rear = 0;  /* one slot kept empty */`
		
		`void enqueue(int x) {`
		`  if ((rear + 1) % MAX == front) { printf("Full\n"); return; }`
		`  q[rear] = x;`
		`  rear = (rear + 1) % MAX;`
		`}`
		
		`int dequeue(void) {`
		`  if (front == rear) { printf("Empty\n"); return -1; }`
		`  int x = q[front];`
		`  front = (front + 1) % MAX;`
		`  return x;`
		`}`

24	**Write the enqueue and dequeue routines for a linked-list queue.**	`struct Node { int data; struct Node *next; };`
		`struct Node *front = NULL, *rear = NULL;`
		
		`void enqueue(int x) {`
		`  struct Node *n = malloc(sizeof(struct Node));`
		`  n->data = x;  n->next = NULL;`
		`  if (rear == NULL) front = rear = n;  /* first element */`
		`  else { rear->next = n; rear = n; }`
		`}`
		
		`int dequeue(void) {`
		`  if (front == NULL) { printf("Queue empty\n"); return -1; }`
		`  struct Node *t = front;`
		`  int x = t->data;`
		`  front = front->next;`
		`  if (front == NULL) rear = NULL;  /* queue became empty */`
		`  free(t);`
		`  return x;`
		`}`

25	**What is a double-ended queue (deque)? What are its variants?**	A deque allows insertion and deletion at **both ends**. • **Input-restricted deque:** insertion at one end only, deletion at both ends. • **Output-restricted deque:** deletion at one end only, insertion at both ends. A deque can behave as a stack or as a queue. It is used in sliding-window problems and work-stealing schedulers.
26	**What is a priority queue? What are its operations and implementations?**	A priority queue is an abstract data type in which every element has a priority and the element with the **highest (or lowest) priority is served first**, irrespective of the order of arrival. Elements with equal priority are served in FIFO order. • **Operations:** Insert, DeleteMin (or DeleteMax), FindMin (or FindMax). • **Implementations:** unsorted array/list (insert O(1), delete O(n)); sorted list (insert O(n), delete O(1)); **binary heap** (both O(log n)) - the usual choice.
27	**List some applications of queues.**	• CPU scheduling (round robin) and disk scheduling. • Printer spooling and I/O buffers. • Breadth-first search in graphs and level-order traversal of trees. • Handling requests in web servers and call centres. • Simulation of real-world waiting lines.
28	**List some applications of priority queues.**	• Dijkstra's shortest-path and Prim's minimum-spanning-tree algorithms. • Huffman coding (greedy algorithms). • Operating-system job scheduling by priority. • Event-driven (discrete-event) simulation. • External sorting and finding the k largest or smallest elements.

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>29</td><td>Differentiate between a stack and a queue.</td><td>
<table>
<tr><th>Stack</th><th>Queue</th></tr>
<tr><td>LIFO</td><td>FIFO</td></tr>
<tr><td>Insertion and deletion at the same end (top)</td><td>Insertion at rear, deletion at front</td></tr>
<tr><td>One pointer (top) is enough</td><td>Two pointers (front and rear)</td></tr>
<tr><td>Full when top == MAX - 1</td><td>Full when rear == MAX - 1 (simple queue)</td></tr>
<tr><td>Used in recursion, undo, expression evaluation</td><td>Used in scheduling, buffering, BFS</td></tr>
</table>
</td></tr>
<tr><td>30</td><td>How can a queue be implemented using two stacks?</td><td>Keep an input stack and an output stack. Enqueue pushes onto the input stack. Dequeue pops from the output stack; if the output stack is empty, first pop everything from the input stack and push it onto the output stack, which reverses the order so the oldest element is on top. Each element moves at most twice, so the amortised cost of every operation is O(1).</td></tr>
<tr><td>31</td><td>How do you test for an empty stack and an empty queue?</td><td>
<table>
<tr><th>Structure</th><th>Empty condition</th></tr>
<tr><td>Array stack (top starts at -1)</td><td>top == -1</td></tr>
<tr><td>Linked stack</td><td>top == NULL</td></tr>
<tr><td>Simple array queue (front = 0, rear = -1)</td><td>front > rear</td></tr>
<tr><td>Circular queue (one slot unused)</td><td>front == rear</td></tr>
<tr><td>Linked queue</td><td>front == NULL</td></tr>
</table>
</td></tr>
</table>

UNIT 3 - Trees

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>What is a non-linear data structure?</td><td>A non-linear data structure can express relationships richer than simple 'before and after'. An element may be connected to several others, so the elements cannot be laid out in a single sequence. Trees and graphs are the two main examples.</td></tr>
<tr><td>2</td><td>Define a tree.</td><td>A tree is a connected, acyclic data structure that represents a hierarchy. It consists of a special node called the root and zero or more disjoint sub-trees, each of which is itself a tree. A tree with n nodes has exactly n - 1 edges, and there is exactly one path from the root to every node.</td></tr>
<tr><td>3</td><td>Explain the basic tree terminology.</td><td>
<table>
<tr><th>Term</th><th>Meaning</th></tr>
<tr><td>Root</td><td>Topmost node; has no parent</td></tr>
<tr><td>Parent / Child</td><td>A node directly above / below another node</td></tr>
<tr><td>Siblings</td><td>Nodes that share the same parent</td></tr>
<tr><td>Leaf (external node)</td><td>A node with no children</td></tr>
<tr><td>Internal node</td><td>A node with at least one child</td></tr>
<tr><td>Degree of a node</td><td>Number of children of the node</td></tr>
<tr><td>Level / Depth</td><td>Number of edges from the root to the node (root is at level 0)</td></tr>
<tr><td>Height of a node</td><td>Number of edges on the longest path down to a leaf</td></tr>
<tr><td>Height of a tree</td><td>Height of the root</td></tr>
<tr><td>Path length</td><td>Number of edges on the path between two nodes</td></tr>
<tr><td>Subtree</td><td>A node together with all its descendants</td></tr>
</table>
</td></tr>
<tr><td>4</td><td>What is a binary tree? Give its node structure.</td><td>A binary tree is a tree in which every node has at most two children, called the left child and the right child.<pre><code>struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
};</code></pre></td></tr>
<tr><td>5</td><td>State the important properties of a binary tree.</td><td>

Maximum number of nodes at level i is 2^i.
Maximum number of nodes in a binary tree of height h is $2^{h+1} - 1$.
A binary tree with n nodes has height at least floor($\log_2 n$).
If n_0 is the number of leaves and n_2 the number of nodes with two children, then $\mathbf{n_0 = n_2 + 1}$.

</td></tr>
<tr><td>6</td><td>Differentiate between strictly binary, complete, perfect and skewed binary trees.</td><td>
<table>
<tr><th>Type</th><th>Definition</th></tr>
<tr><td>Strictly (full) binary tree</td><td>Every node has either 0 or 2 children</td></tr>
<tr><td>Perfect binary tree</td><td>Strictly binary and all leaves are at the same level; has $2^{h+1} - 1$ nodes</td></tr>
<tr><td>Complete (almost complete) binary tree</td><td>All levels are completely filled except possibly the last, which is filled from left to right</td></tr>
<tr><td>Skewed binary tree</td><td>Every node has only a left child (left-skewed) or only a right child (right-skewed); behaves like a linked list</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>7</td><td>In how many ways can a binary tree be represented?</td><td>

Array (sequential) representation: for a node stored at index i (0-based), its left child is at 2i + 1, its right child at 2i + 2 and its parent at (i - 1) / 2. It is efficient only for complete trees; skewed trees waste a lot of space.
Linked representation: each node holds data and two pointers (left, right). It is flexible and uses memory proportional to the number of nodes.

</td></tr>
<tr><td>8</td><td>What is tree traversal? Name the types.</td><td>
Traversal means visiting every node of the tree exactly once in a systematic order.

Preorder: Root, Left, Right
Inorder: Left, Root, Right
Postorder: Left, Right, Root
Level order: level by level, using a queue

</td></tr>
<tr><td>9</td><td>Write the recursive routines for the three depth-first traversals.</td><td>
<pre>
void preorder(struct TreeNode *t) {
    if (t == NULL) return;
    printf("%d ", t->data);
    preorder(t->left);
    preorder(t->right);
}

void inorder(struct TreeNode *t) {
    if (t == NULL) return;
    inorder(t->left);
    printf("%d ", t->data);
    inorder(t->right);
}

void postorder(struct TreeNode *t) {
    if (t == NULL) return;
    postorder(t->left);
    postorder(t->right);
    printf("%d ", t->data);
}
</pre>
Each runs in O(n) time; the recursion stack uses O(h) space.
</td></tr>
<tr><td>10</td><td>Give the traversals of the following tree.</td><td>
<pre>
        A
       / \
      B   C
     / \   \
    D   E   F
</pre>
<table>
<tr><th>Traversal</th><th>Output</th></tr>
<tr><td>Preorder</td><td>A B D E C F</td></tr>
<tr><td>Inorder</td><td>D B E A C F</td></tr>
<tr><td>Postorder</td><td>D E B F C A</td></tr>
<tr><td>Level order</td><td>A B C D E F</td></tr>
</table>
</td></tr>
<tr><td>11</td><td>Can a binary tree be built from its traversals?</td><td>Yes, if inorder is given together with either preorder or postorder. The first element of preorder (or last of postorder) is the root; locating it in the inorder sequence splits the nodes into the left and right subtrees, and the process repeats. Preorder and postorder alone are not enough to rebuild a unique tree.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>12</td><td>What is an expression tree? Draw the tree and give the prefix and postfix forms of a + ((b * (c - e)) / f).</td><td>In an expression tree the leaves are operands and the internal nodes are operators.<pre>
        +
       / \
      a   /
         / \
        *   f
       / \
      b   -
         / \
        c   e
</pre><table>
<tr><th>Traversal</th><th>Result</th></tr>
<tr><td>Preorder (prefix)</td><td>+ a / * b - c e f</td></tr>
<tr><td>Inorder (infix)</td><td>a + b * c - e / f (brackets needed)</td></tr>
<tr><td>Postorder (postfix)</td><td>a b c e - * f / +</td></tr>
</table></td></tr>
<tr><td>13</td><td>Define a binary search tree (BST).</td><td>A BST is a binary tree in which, for every node x, all keys in the left subtree are smaller than x and all keys in the right subtree are larger than x (duplicates are either disallowed or placed consistently on one side). The inorder traversal of a BST gives the keys in sorted order.</td></tr>
<tr><td>14</td><td>Write the search and insert routines for a BST.</td><td><pre>
struct TreeNode *search(struct TreeNode *t, int key) {
    if (t == NULL || t->data == key) return t;
    if (key < t->data) return search(t->left, key);
    return search(t->right, key);
}

struct TreeNode *insert(struct TreeNode *t, int key) {
    if (t == NULL) {
        t = malloc(sizeof(struct TreeNode));
        t->data = key;
        t->left = t->right = NULL;
    }
    else if (key < t->data) t->left = insert(t->left, key);
    else if (key > t->data) t->right = insert(t->right, key);
    return t;
}
</pre>Average time O(log n); worst case O(n) when the tree is skewed (for example, keys inserted in sorted order).</td></tr>
<tr><td>15</td><td>Explain the steps of deleting a node from a BST.</td><td>1. Node is a leaf: simply remove it.
2. Node has one child: replace the node by its child.
3. Node has two children: find its inorder successor (smallest node of the right subtree) or inorder predecessor (largest node of the left subtree), copy that value into the node, and then delete the successor/predecessor, which has at most one child.</td></tr>
<tr><td>16</td><td>How do you find the minimum and the maximum key in a BST?</td><td>The minimum is found by following left links until a node has no left child. The maximum is found by following right links until a node has no right child. Both take O(h) time.</td></tr>
<tr><td>17</td><td>What is a threaded binary tree? What is its advantage?</td><td>In an ordinary binary tree about half of the pointers are NULL. In a threaded binary tree these NULL pointers are replaced by threads: a NULL right pointer points to the node's inorder successor and a NULL left pointer points to its inorder predecessor.
• Inorder traversal can be done without recursion and without a stack.
• It is a single-threaded tree if only one kind of thread is used, and double-threaded if both are used.
• An extra flag per pointer tells whether it is a real child link or a thread.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>18</td><td>What is an AVL tree? What is a balance factor?</td><td>An AVL tree is a self-balancing BST in which, for every node, the heights of the left and right subtrees differ by at most 1.
Balance factor of a node = height(left subtree) - height(right subtree). In an AVL tree it must be -1, 0 or +1 for every node. Because of this, the height is always O(log n), so search, insert and delete are guaranteed O(log n).</td></tr>
<tr><td>19</td><td>Explain the rotations used to rebalance an AVL tree.</td><td><table><tr><th>Case</th><th>Cause</th><th>Fix</th></tr><tr><td>LL</td><td>Insertion in the left subtree of the left child</td><td>Single right rotation</td></tr><tr><td>RR</td><td>Insertion in the right subtree of the right child</td><td>Single left rotation</td></tr><tr><td>LR</td><td>Insertion in the right subtree of the left child</td><td>Left rotation on the child, then right rotation on the node</td></tr><tr><td>RL</td><td>Insertion in the left subtree of the right child</td><td>Right rotation on the child, then left rotation on the node</td></tr></table>The node closest to the new leaf whose balance factor becomes +2 or -2 is called the pivot; the rotation is applied at the pivot.</td></tr>
<tr><td>20</td><td>What is a heap? Differentiate min-heap and max-heap.</td><td>A heap is a complete binary tree that satisfies the heap-order property, usually stored in an array.<table><tr><th>Max-heap</th><th>Min-heap</th></tr><tr><td>Every parent is greater than or equal to its children</td><td>Every parent is less than or equal to its children</td></tr><tr><td>Root holds the maximum</td><td>Root holds the minimum</td></tr></table>A complete binary tree of height h has between 2^h and 2^{h+1} - 1 nodes, so the height of a heap of n nodes is floor($\log_2$ n).</td></tr>
<tr><td>21</td><td>Explain insertion and deletion in a binary heap.</td><td>Insert: place the new element at the next free position (end of the array) and percolate up: while it is smaller than its parent (min-heap), swap them. Time O(log n).
DeleteMin: remove the root, move the last element to the root and percolate down: swap it with the smaller child until the heap order is restored. Time O(log n).
Build-heap from n elements can be done in O(n) time by percolating down from the last internal node to the root.</td></tr>
<tr><td>22</td><td>How is a heap used to implement a priority queue?</td><td>A priority queue needs fast insert and fast removal of the highest-priority element. A binary heap provides both in O(log n), whereas a sorted list or an unsorted list makes one of the two operations O(n). Also, unlike a BST, a heap never becomes skewed because it is always complete.</td></tr>
<tr><td>23</td><td>How is a general tree converted into a binary tree?</td><td>Use the left-child, right-sibling representation:
The root of the general tree becomes the root of the binary tree.
The left pointer of a node points to its first (leftmost) child.
The right pointer of a node points to its next sibling.
Repeat for every node.
Every general tree (or forest) can be represented as a unique binary tree this way.</td></tr>
<tr><td>24</td><td>What is a directed tree and an ordered tree?</td><td>A directed (rooted) tree is an acyclic directed graph with one node, the root, of in-degree 0, while all other nodes have in-degree 1. An ordered tree is a rooted tree in which the order of the children of each node is significant.</td></tr>
<tr><td>25</td><td>What is the length of a path in a tree?</td><td>The length of a path is the number of edges on it. In a tree there is exactly one path from the root to each node, and the depth of a node equals the length of that path.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>26</td><td>Compare a binary tree and a binary search tree.</td><td>
<table>
<tr><th>Binary tree</th><th>Binary search tree</th></tr>
<tr><td>No ordering rule on the keys</td><td>Left subtree < node < right subtree</td></tr>
<tr><td>Search needs O(n)</td><td>Search needs O(h), O(log n) on average</td></tr>
<tr><td>Inorder gives no special order</td><td>Inorder gives sorted order</td></tr>
</table>
</td></tr>
<tr><td>27</td><td>Give the time complexities of search, insert and delete in BST, AVL tree and heap.</td><td>
<table>
<tr><th>Structure</th><th>Search</th><th>Insert</th><th>Delete</th></tr>
<tr><td>BST (average / worst)</td><td>O(log n) / O(n)</td><td>O(log n) / O(n)</td><td>O(log n) / O(n)</td></tr>
<tr><td>AVL tree (worst)</td><td>O(log n)</td><td>O(log n)</td><td>O(log n)</td></tr>
<tr><td>Binary heap</td><td>O(n)</td><td>O(log n)</td><td>O(log n) for root; find min O(1)</td></tr>
</table>
</td></tr>
<tr><td>28</td><td>List the applications of trees.</td><td>

File-system and folder hierarchy; organisation charts; XML/HTML DOM.
Expression parsing and evaluation in compilers.
Searching and indexing: BSTs, AVL trees, B-trees in databases.
Priority queues and heap sort (heaps).
Huffman coding for data compression; decision trees; routing tables (tries).

</td></tr>
</table>

UNIT 4 - Graphs

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>Define a graph.</td><td>A graph G = (V, E) consists of a non-empty set V of vertices (nodes) and a set E of edges, where each edge connects a pair of vertices. Graphs model networks such as roads, social connections and the internet.</td></tr>
<tr><td>2</td><td>Differentiate between a directed and an undirected graph.</td><td>
<table>
<tr><th>Directed graph (digraph)</th><th>Undirected graph</th></tr>
<tr><td>Every edge has a direction: (u, v) goes from u to v</td><td>Edges have no direction: (u, v) and (v, u) are the same</td></tr>
<tr><td>Drawn with arrows</td><td>Drawn with plain lines</td></tr>
<tr><td>Vertices have in-degree and out-degree</td><td>Vertices have a single degree</td></tr>
<tr><td>Example: web links, one-way streets</td><td>Example: friendships, two-way roads</td></tr>
</table>
</td></tr>
<tr><td>3</td><td>Explain the basic graph terminology.</td><td>
<table>
<tr><th>Term</th><th>Meaning</th></tr>
<tr><td>Adjacent vertices</td><td>Two vertices joined directly by an edge</td></tr>
<tr><td>Degree</td><td>Number of edges incident on a vertex (undirected)</td></tr>
<tr><td>In-degree / Out-degree</td><td>Number of edges coming into / going out of a vertex (directed)</td></tr>
<tr><td>Loop (self-loop)</td><td>An edge that joins a vertex to itself</td></tr>
<tr><td>Parallel edges</td><td>Two or more edges between the same pair of vertices</td></tr>
<tr><td>Simple graph</td><td>No loops and no parallel edges</td></tr>
<tr><td>Weighted graph</td><td>A number (cost, distance) is attached to every edge</td></tr>
<tr><td>Path</td><td>A sequence of vertices in which consecutive vertices are adjacent</td></tr>
<tr><td>Simple path</td><td>A path in which no vertex is repeated</td></tr>
<tr><td>Cycle</td><td>A path that starts and ends at the same vertex</td></tr>
<tr><td>Acyclic graph</td><td>A graph with no cycles; a directed one is a DAG</td></tr>
<tr><td>Complete graph</td><td>Every pair of distinct vertices is joined by an edge</td></tr>
<tr><td>Subgraph</td><td>A graph whose vertices and edges are subsets of those of G</td></tr>
</table>
</td></tr>
<tr><td>4</td><td>What is a connected graph? Explain strongly connected and weakly connected graphs.</td><td>

Connected (undirected): there is a path between every pair of vertices.
Strongly connected (directed): there is a directed path from every vertex to every other vertex.
Weakly connected (directed): the digraph is not strongly connected, but it becomes connected if the directions of the edges are ignored.

</td></tr>
<tr><td>5</td><td>What is a biconnected graph? What is an articulation point?</td><td>A connected undirected graph is biconnected if removing any single vertex still leaves the graph connected. A vertex whose removal disconnects the graph is an articulation point (cut vertex). A graph is biconnected exactly when it has no articulation points. Such graphs are important in designing fault-tolerant networks; articulation points are found with a modified DFS in O(V + E).</td></tr>
<tr><td>6</td><td>What is the maximum number of edges in a simple graph with n vertices?</td><td>Undirected: n(n - 1) / 2. Directed: n(n - 1).
Proof (undirected): every one of the n vertices can be joined to the other n - 1, which counts each edge twice, so the maximum is n(n - 1) / 2. Equivalently, (n - 1) + (n - 2) + ... + 1 = n(n - 1) / 2.
Related fact (handshaking lemma): the sum of the degrees of all vertices equals 2|E|.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>7</td><td>In how many ways can a graph be represented?</td><td>1. Adjacency matrix - a V x V matrix where entry [i][j] is 1 (or the edge weight) if there is an edge from i to j, else 0.
2. Adjacency list - an array of V lists; the list at index i holds all vertices adjacent to i.
3. Edge list and incidence matrix are less common alternatives.</td></tr>
<tr><td>8</td><td>Compare adjacency matrix and adjacency list.</td><td><table><tr><th>Property</th><th>Adjacency matrix</th><th>Adjacency list</th></tr><tr><td>Space</td><td>$O(V^2)$</td><td>O(V + E)</td></tr><tr><td>Check if edge (u, v) exists</td><td>O(1)</td><td>O(degree of u)</td></tr><tr><td>List all neighbours of u</td><td>O(V)</td><td>O(degree of u)</td></tr><tr><td>Best for</td><td>Dense graphs</td><td>Sparse graphs</td></tr></table></td></tr>
<tr><td>9</td><td>Give the adjacency matrix and adjacency list of this undirected graph: edges A-B, A-C, B-D, C-D.</td><td><pre>Matrix (order A B C D):
    A B C D
 A 0 1 1 0
 B 1 0 0 1
 C 1 0 0 1
 D 0 1 1 0

List:
A : B, C
B : A, D
C : A, D
D : B, C</pre>For an undirected graph the matrix is symmetric.</td></tr>
<tr><td>10</td><td>What are the two graph traversal methods?</td><td>• Breadth-First Search (BFS) - visits vertices level by level; uses a queue.
• Depth-First Search (DFS) - goes as deep as possible along a path before backtracking; uses a stack or recursion.
Both take O(V + E) time with an adjacency list and $O(V^2)$ with an adjacency matrix.</td></tr>
<tr><td>11</td><td>Explain the BFS algorithm.</td><td>1. Mark the start vertex as visited and enqueue it.
2. While the queue is not empty: dequeue a vertex u and visit it.
3. For every unvisited neighbour v of u, mark it visited and enqueue it.
BFS finds the shortest path in terms of number of edges in an unweighted graph. Applications: shortest hops, finding connected components, web crawling, level-wise processing.</td></tr>
<tr><td>12</td><td>Explain the DFS algorithm.</td><td><pre>void DFS(int u) {
    visited[u] = 1;
    printf("%d ", u);
    for (each neighbour v of u)
        if (!visited[v])
            DFS(v);
}</pre>Applications: detecting cycles, topological sorting, finding connected components, strongly connected components, articulation points, solving mazes.</td></tr>
<tr><td>13</td><td>Write the BFS and DFS orders for the graph in the previous example (start at A, neighbours in alphabetical order).</td><td><table><tr><th>Method</th><th>Order of visit</th></tr><tr><td>BFS</td><td>A, B, C, D</td></tr><tr><td>DFS</td><td>A, B, D, C</td></tr></table></td></tr>
<tr><td>14</td><td>What is the difference between BFS and DFS?</td><td><table><tr><th>BFS</th><th>DFS</th></tr><tr><td>Uses a queue</td><td>Uses a stack / recursion</td></tr><tr><td>Explores neighbours level by level</td><td>Explores one branch fully before backtracking</td></tr><tr><td>Finds shortest path in unweighted graphs</td><td>Does not guarantee shortest path</td></tr><tr><td>Needs more memory for wide graphs</td><td>Needs more memory for very deep graphs</td></tr></table></td></tr>
</table>

S.No	Question	Answer
15	**What is a spanning tree? What is a minimum spanning tree (MST)?**	A **spanning tree** of a connected graph G is a subgraph that contains all the vertices of G, is connected and has no cycles; it therefore has exactly V - 1 edges. In a weighted graph, the spanning tree whose **total edge weight is the smallest** is the minimum spanning tree. Prim's and Kruskal's algorithms find it.
16	**Explain Kruskal's algorithm.**	1. Sort all edges in non-decreasing order of weight. 2. Pick the next smallest edge. If adding it does **not form a cycle** (checked with Union-Find / disjoint sets), include it in the MST; otherwise discard it. 3. Stop when V - 1 edges have been chosen. It is a **greedy** algorithm. Time: O(E log E). It works well on sparse graphs.
17	**Explain Prim's algorithm.**	1. Start from any vertex and mark it as part of the tree. 2. Among all edges that connect a tree vertex with a non-tree vertex, choose the one with the smallest weight and add it (and its new vertex) to the tree. 3. Repeat until all vertices are in the tree. Also greedy. Time: $O(V^2)$ with a matrix, O(E log V) with a binary heap. It suits dense graphs.
18	**What is the shortest path problem? What is Dijkstra's algorithm used for?**	Given a weighted graph and a source vertex, the **single-source shortest path problem** asks for the minimum-weight path from the source to every other vertex (path length = sum of edge weights). **Dijkstra's algorithm** solves it when all weights are **non-negative**. It keeps a distance estimate for every vertex, repeatedly picks the unvisited vertex with the smallest distance, and **relaxes** its outgoing edges. Time: $O(V^2)$ or O((V + E) log V) with a priority queue.
19	**When does Dijkstra's algorithm fail? Which algorithms handle such cases?**	Dijkstra's algorithm can give wrong answers when edges have **negative weights**. **Bellman-Ford** (O(VE)) handles negative edges and detects negative cycles. **Floyd-Warshall** ($O(V^3)$) finds shortest paths between **all pairs** of vertices.
20	**What is topological sorting?**	A topological sort is a linear ordering of the vertices of a **directed acyclic graph (DAG)** such that for every edge (u, v), vertex u appears **before** v. It is possible only if the graph has no cycle. 1. Compute the in-degree of every vertex. 2. Put all vertices with in-degree 0 in a queue. 3. Remove a vertex from the queue, output it, and reduce the in-degree of its neighbours; add any that reach 0 to the queue. 4. If fewer than V vertices were output, the graph has a cycle. Time O(V + E). Uses: task scheduling, build systems, course prerequisites.
21	**How can a cycle be detected in a graph?**	In an **undirected** graph a cycle exists if DFS reaches an already visited vertex that is not the parent of the current vertex (or with Union-Find). In a **directed** graph a cycle exists if DFS finds a **back edge**, i.e. an edge to a vertex that is still on the current recursion stack, or if topological sort cannot output all vertices.
22	**What do visiting and traversing mean in a graph?**	• **Visiting** means processing a vertex (for example, reading or printing its value) when control reaches it. • **Traversing** means moving through all the vertices in some systematic order such as BFS or DFS.

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>23</td><td>Name the basic operations on a graph.</td><td>• Add vertex, remove vertex
• Add edge, remove edge
• Check adjacency of two vertices; find the neighbours of a vertex
• Traversal (BFS, DFS)</td></tr>
<tr><td>24</td><td>What is the difference between a tree and a graph?</td><td><table><tr><th>Tree</th><th>Graph</th></tr><tr><td>Connected and acyclic</td><td>May have cycles and may be disconnected</td></tr><tr><td>Exactly one path between any two nodes</td><td>May have many paths, or none</td></tr><tr><td>n nodes, n - 1 edges</td><td>Any number of edges</td></tr><tr><td>Has a root (hierarchy)</td><td>No root; no hierarchy</td></tr></table></td></tr>
<tr><td>25</td><td>What is the class P, the class NP and an NP-complete problem?</td><td>• P - decision problems that can be solved in polynomial time.
• NP - decision problems for which a proposed solution can be verified in polynomial time.
• NP-complete - problems that are in NP and to which every problem in NP can be reduced in polynomial time. If one NP-complete problem had a polynomial-time algorithm, then P = NP.
Whether P = NP is an open problem.</td></tr>
<tr><td>26</td><td>Give some examples of NP-complete decision problems.</td><td>• Boolean satisfiability (SAT) - the first problem proved NP-complete (Cook's theorem).
• Hamiltonian cycle and the decision version of the Travelling Salesman Problem.
• Vertex cover, clique, and independent set.
• Graph colouring with 3 or more colours, subset sum, and the 0/1 knapsack decision problem.</td></tr>
<tr><td>27</td><td>List some applications of graphs.</td><td>• Maps and navigation (shortest path); network routing.
• Social networks (friend recommendations); web page ranking.
• Dependency and scheduling problems (topological sort).
• Circuit design; telecom and electricity network layout (MST).
• Compilers (control-flow graphs); recommendation systems.</td></tr>
</table>

UNIT 5 - Sorting, Searching and Hashing

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>What is sorting? Why is it needed?</td><td>Sorting is arranging the elements of a list in a particular order, such as ascending or descending, numerically or alphabetically. Sorted data makes searching (binary search), merging, duplicate detection and reporting much faster and easier.</td></tr>
<tr><td>2</td><td>What is the difference between internal and external sorting?</td><td>
<table>
<tr><th>Internal sorting</th><th>External sorting</th></tr>
<tr><td>All the data fits in main memory (RAM)</td><td>Data is too large for main memory and stays on disk or tape</td></tr>
<tr><td>Examples: bubble, insertion, quick, heap sort</td><td>Example: external merge sort (sort chunks, then merge the sorted runs)</td></tr>
<tr><td>Cost is measured by comparisons and swaps</td><td>Cost is dominated by the number of disk reads and writes</td></tr>
</table>
</td></tr>
<tr><td>3</td><td>What factors decide the choice of a sorting algorithm?</td><td>

Size of the input and how it is already ordered (random, nearly sorted, reversed).
Time complexity: best, average and worst case.
Extra memory needed (in-place or not).
Whether the sort must be stable.
Simplicity of coding and the programmer's familiarity.

</td></tr>
<tr><td>4</td><td>What do 'stable' and 'in-place' sorting mean?</td><td>A sort is stable if records with equal keys keep their original relative order after sorting (important when sorting by one column after another). A sort is in-place if it needs only O(1) (or very little) extra memory besides the input array.</td></tr>
<tr><td>5</td><td>Explain bubble sort and write its routine. Why is it called bubble sort?</td><td>Bubble sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass the largest remaining element 'bubbles up' to its final position at the end of the array.
<pre><code>void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = 1;
            }
        }
        if (!swapped) break;   /* already sorted */
    }
}</code></pre>
Time: $O(n^2)$ average and worst, O(n) best (with the swapped flag). Stable and in-place.</td></tr>
<tr><td>6</td><td>Explain selection sort and write its routine.</td><td>In each pass, find the smallest element in the unsorted part and swap it with the first element of that part. After pass i, the first i positions hold the i smallest elements.
<pre><code>void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        int t = a[i]; a[i] = a[min]; a[min] = t;
    }
}</code></pre>
Time: $O(n^2)$ in all cases, but only O(n) swaps. In-place; not stable in the usual form.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>7</td><td>Explain insertion sort and write its routine. How many passes does it need?</td><td>Insertion sort builds the sorted part one element at a time: it takes the next element and inserts it into its correct place among the already sorted elements on its left, shifting larger elements one position to the right. For n elements it needs n - 1 passes; after pass p, positions 0 to p are sorted.<pre>void insertionSort(int a[], int n) {
    for (int p = 1; p < n; p++) {
        int tmp = a[p], j;
        for (j = p; j > 0 && a[j - 1] > tmp; j--)
            a[j] = a[j - 1];
        a[j] = tmp;
    }
}</pre>Time: O(n^2) average and worst, O(n) best (already sorted). Stable, in-place, and the best choice for small or nearly sorted data.</td></tr>
<tr><td>8</td><td>Show the passes of insertion sort on 5, 2, 4, 6, 1, 3.</td><td><table>
<tr><th>Pass</th><th>Array after the pass</th></tr>
<tr><td>Start</td><td>5 2 4 6 1 3</td></tr>
<tr><td>1</td><td>2 5 4 6 1 3</td></tr>
<tr><td>2</td><td>2 4 5 6 1 3</td></tr>
<tr><td>3</td><td>2 4 5 6 1 3</td></tr>
<tr><td>4</td><td>1 2 4 5 6 3</td></tr>
<tr><td>5</td><td>1 2 3 4 5 6</td></tr>
</table></td></tr>
<tr><td>9</td><td>Explain merge sort.</td><td>Merge sort is a divide and conquer algorithm: split the array into two halves, sort each half recursively, then merge the two sorted halves into one sorted array using a temporary array.<pre>void mergeSort(int a[], int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergeSort(a, lo, mid);
    mergeSort(a, mid + 1, hi);
    merge(a, lo, mid, hi);   /* combine two sorted halves */
}</pre>Time: O(n log n) in best, average and worst cases. Extra space O(n). Stable. Well suited to linked lists and to external sorting.</td></tr>
<tr><td>10</td><td>Explain quick sort.</td><td>Quick sort is also divide and conquer. Choose a pivot, partition the array so that elements smaller than the pivot lie to its left and larger ones to its right (the pivot is now in its final position), and then sort the two parts recursively.<pre>int partition(int a[], int lo, int hi) {
    int pivot = a[hi], i = lo - 1;
    for (int j = lo; j < hi; j++)
        if (a[j] <= pivot) {
            i++;
            int t = a[i]; a[i] = a[j]; a[j] = t;
        }
    int t = a[i + 1]; a[i + 1] = a[hi]; a[hi] = t;
    return i + 1;
}

void quickSort(int a[], int lo, int hi) {
    if (lo < hi) {
        int p = partition(a, lo, hi);
        quickSort(a, lo, p - 1);
        quickSort(a, p + 1, hi);
    }
}</pre>Time: O(n log n) average, O(n^2) worst (bad pivots, e.g. an already sorted array with the last element as pivot). In-place, not stable. Choosing a random or median-of-three pivot makes the worst case unlikely.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>11</td><td>Explain heap sort.</td><td>1. Build a max-heap from the array in O(n).
2. Swap the root (largest) with the last element of the heap and reduce the heap size by one.
3. Percolate the new root down to restore the heap property.
4. Repeat until the heap has one element.
Time: O(n log n) in all cases. In-place, not stable.</td></tr>
<tr><td>12</td><td>Explain shell sort.</td><td>Shell sort is a generalised insertion sort that first compares and moves elements that are far apart (a gap h), then repeatedly reduces the gap until it becomes 1, when it finishes as ordinary insertion sort. By then the array is almost sorted, so the last pass is fast. Running time depends on the gap sequence; it is between O(n log n) and $O(n^2)$. In-place, not stable.</td></tr>
<tr><td>13</td><td>Explain radix sort.</td><td>Radix sort is a non-comparison sort. It sorts the numbers digit by digit, from the least significant digit to the most significant, using a stable sort (such as counting sort or buckets) on each digit. With d digits and base b, the time is O(d (n + b)). It works for integers and fixed-length strings.</td></tr>
<tr><td>14</td><td>Compare the common sorting algorithms.</td><td><table>
<tr><th>Algorithm</th><th>Best</th><th>Average</th><th>Worst</th><th>Extra space</th><th>Stable</th></tr>
<tr><td>Bubble</td><td>O(n)</td><td>$O(n^2)$</td><td>$O(n^2)$</td><td>O(1)</td><td>Yes</td></tr>
<tr><td>Selection</td><td>$O(n^2)$</td><td>$O(n^2)$</td><td>$O(n^2)$</td><td>O(1)</td><td>No</td></tr>
<tr><td>Insertion</td><td>O(n)</td><td>$O(n^2)$</td><td>$O(n^2)$</td><td>O(1)</td><td>Yes</td></tr>
<tr><td>Merge</td><td>O(n log n)</td><td>O(n log n)</td><td>O(n log n)</td><td>O(n)</td><td>Yes</td></tr>
<tr><td>Quick</td><td>O(n log n)</td><td>O(n log n)</td><td>$O(n^2)$</td><td>O(log n)</td><td>No</td></tr>
<tr><td>Heap</td><td>O(n log n)</td><td>O(n log n)</td><td>O(n log n)</td><td>O(1)</td><td>No</td></tr>
<tr><td>Radix</td><td>O(d(n+b))</td><td>O(d(n+b))</td><td>O(d(n+b))</td><td>O(n+b)</td><td>Yes</td></tr>
</table></td></tr>
<tr><td>15</td><td>What is the lower bound for comparison-based sorting?</td><td>Any algorithm that sorts only by comparing elements needs at least O(n log n) comparisons in the worst case, because a decision tree that distinguishes all n! orderings has height at least $\log_2(n!)$, which is of the order n log n. Radix and counting sort avoid this bound because they do not compare elements.</td></tr>
<tr><td>16</td><td>Which sorting algorithm would you choose in the following situations?</td><td><table>
<tr><th>Situation</th><th>Suitable choice</th></tr>
<tr><td>Small array or nearly sorted data</td><td>Insertion sort</td></tr>
<tr><td>Need guaranteed O(n log n) and stability</td><td>Merge sort</td></tr>
<tr><td>Need fast average speed, in-place</td><td>Quick sort</td></tr>
<tr><td>Need O(n log n) worst case and no extra memory</td><td>Heap sort</td></tr>
<tr><td>Data larger than RAM</td><td>External merge sort</td></tr>
<tr><td>Integers with few digits</td><td>Radix sort</td></tr>
</table></td></tr>
<tr><td>17</td><td>What is searching? Name the types of searching.</td><td>Searching is finding whether a given key is present in a collection, and if so, where. The search is successful if the key is found and unsuccessful otherwise. The two basic techniques are linear (sequential) search and binary search.</td></tr>
<tr><td>18</td><td>Explain linear search and write its routine.</td><td>Linear search compares the key with each element one by one from the beginning until a match is found or the list ends. It works on unsorted data.
<pre>int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
}</pre>Time: O(1) best, O(n) average and worst. Space: O(1).</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>19</td><td>Explain binary search and write its routine.</td><td>Binary search works only on a sorted array. Compare the key with the middle element: if equal, stop; if the key is smaller, search the left half; otherwise search the right half. The search range is halved at every step.<pre>int binarySearch(int a[], int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == key) return mid;
        else if (key < a[mid]) hi = mid - 1;
        else lo = mid + 1;
    }
    return -1;
}</pre>Time: O(1) best, O(log n) average and worst. Space O(1) for the iterative form.</td></tr>
<tr><td>20</td><td>Compare linear search and binary search.</td><td><table><tr><th>Linear search</th><th>Binary search</th></tr><tr><td>Works on sorted or unsorted data</td><td>Needs sorted data</td></tr><tr><td>O(n)</td><td>O(log n)</td></tr><tr><td>Works on arrays and linked lists</td><td>Needs random access (array)</td></tr><tr><td>Simple; good for small lists</td><td>Much faster for large lists</td></tr></table></td></tr>
<tr><td>21</td><td>What is hashing? Explain the terms hash function, hash table and collision.</td><td>Hashing is a technique that maps a key directly to a position in a table so that search, insert and delete take O(1) time on average.
Hash function h(key) converts the key into a table index, e.g. h(key) = key mod m, where m is the table size (preferably a prime).
Hash table is the array in which records are stored at the computed positions.
Collision occurs when two different keys give the same index.</td></tr>
<tr><td>22</td><td>Explain the methods of collision resolution.</td><td><table><tr><th>Method</th><th>How it works</th></tr><tr><td>Separate chaining</td><td>Every table slot holds a linked list of all keys that hash to it</td></tr><tr><td>Linear probing</td><td>If slot i is taken, try i+1, i+2, ... (mod m); can cause primary clustering</td></tr><tr><td>Quadratic probing</td><td>Try $i+1^2$, $i+2^2$, ... (mod m); reduces primary clustering</td></tr><tr><td>Double hashing</td><td>Use a second hash function h2 to decide the step size: i + k * h2(key)</td></tr></table>Chaining is called closed addressing; probing methods are called open addressing.</td></tr>
<tr><td>23</td><td>What is load factor and rehashing?</td><td>Load factor = number of stored keys / table size. Performance falls as the load factor grows (for open addressing it should be kept below about 0.5 to 0.7). Rehashing means creating a larger table (about twice the size, rounded up to a prime) and re-inserting every key with the new hash function when the load factor crosses a limit.</td></tr>
<tr><td>24</td><td>Insert the keys 23, 43, 13, 27 into a table of size 10 using h(k) = k mod 10 with linear probing.</td><td><table><tr><th>Key</th><th>h(k)</th><th>Final index</th><th>Reason</th></tr><tr><td>23</td><td>3</td><td>3</td><td>free</td></tr><tr><td>43</td><td>3</td><td>4</td><td>slot 3 taken, next is free</td></tr><tr><td>13</td><td>3</td><td>5</td><td>slots 3 and 4 taken</td></tr><tr><td>27</td><td>7</td><td>7</td><td>free</td></tr></table></td></tr>
</table>